

Löb's Theorem and Tiling Agents

The logic of trusting a successor you cannot simulate

Lecture notes, Agent Foundations module
Iliad Intensive

June 2026

Abstract

A sufficiently advanced AI may eventually rewrite its own code or build a successor more capable than itself. This is where a clean and surprising obstacle appears. If the successor is genuinely smarter, the original agent cannot predict exactly what it will do, on pain of being that smart already; so it cannot certify the successor safe by simulating it and watching. It must instead reason *abstractly* about the successor's design, guaranteeing that whatever specific action the successor takes will be acceptable, without knowing which action that is. These notes develop that predicament and the logical wall it runs into. We begin with the Vingean principle, the demand that an agent reason about a more capable mind through general properties rather than move-by-move prediction. We then give a self-contained crash course in the relevant self-reference: Gödel's trick of building self-referential statements out of proof-searching programs, and Löb's theorem, which says that a consistent formal system cannot, in general, prove "if this is provable then it is true." From there the central obstacle, the "Löbstacle," follows almost immediately: a proof-based agent's natural way of trusting a successor that uses the same reasoning is precisely a Löbian statement, so it cannot license that trust without becoming inconsistent, and appears forced to hand each successor a strictly weaker proof system. We show why patching this with a tower of ever-stronger theories fails (the finite descent problem: the tower is well-ordered, so it must bottom out, and each step down is a real loss of mathematical power). We state the three desiderata a self-modifying agent's logic must satisfy (the Vingean, Tiling, and Naturalistic principles), and we close by turning the same self-reference around to show its productive side: Löb's theorem also enables robust mutual cooperation between programs that read each other's code, and it explains a failure mode (spurious counterfactuals) that haunts naive decision algorithms embedded in the worlds they reason about.

Contents

1	Introduction: the trouble with building something smarter	2
2	The Vingean principle: reasoning about a mind you cannot run	2
3	A crash course in self-reference	3
3.1	Gödel's insight, through proof-searching programs	3
3.2	Löb's theorem	3
4	The Löbstacle: a proof-based agent cannot trust its equal	4
5	The finite descent problem	5
6	What self-modifying agents need	6
7	The other face of self-reference	6
7.1	Löbian cooperation	6
7.2	Spurious counterfactuals	7

1 Introduction: the trouble with building something smarter

Imagine we have built Deep Thought 1.0, an AI with a simple and reassuring policy: it reasons about the consequences of its possible actions and only takes an action once it has shown that the action’s consequences are good on balance. We would like this property to be stable. In particular, one action available to Deep Thought 1.0 is to design and switch on Deep Thought 2.0, a successor with sharper deductive abilities. For Deep Thought 1.0 (call it DT_1) to actually build Deep Thought 2.0 (call it DT_2) under its own policy, it must first establish that building DT_2 is good on balance.

Here is the difficulty, and it is worth feeling its full force before we formalize anything. The consequences of building DT_2 include all the actions that DT_2 will go on to take. But DT_2 is, by construction, a better reasoner than DT_1 . So DT_1 cannot in general work out which actions DT_2 will take; if it could, it would already be as capable as its successor, and the successor would be no improvement. The parent is in the position of having to vouch for a child whose conclusions it cannot reach on its own.

Naively the situation looks recoverable. Surely it is enough for DT_1 to know three things: that DT_2 has the same goals it does, that DT_2 ’s reasoning is reliable, and that DT_2 , like itself, only acts on conclusions it has established. From those three facts it seems DT_1 should be able to argue: whatever DT_2 does, it must have shown that doing it is good, and since DT_2 ’s reasoning is reliable, it really is good. Trust achieved, without prediction.

The unwelcome discovery, and the reason a body of MIRI research leans so heavily on mathematical logic, is that the innocent-sounding middle step (“ DT_1 knows that DT_2 ’s reasoning is reliable”) is the one that cannot be taken. When we model the agents as proving statements in formal systems, the only way DT_1 can assert the reliability of a system as strong as its own is if DT_1 (and hence both agents) is in fact *unreliable*. The obstruction is not a failure of cleverness that more computing power would fix. It is a theorem, due to Martin Löb in 1955, about the limits of what a consistent system can prove about its own proofs. The bulk of these notes is an unpacking of why that theorem stands exactly in the path of safe self-improvement, what it does and does not forbid, and what a self-modifying agent’s reasoning would have to look like to get around it.

We proceed as follows. Section 2 states the Vingean principle, which is really the source of the whole problem: a smarter successor must be reasoned about abstractly, never simulated. Section 3 builds the logical machinery from scratch, through proof-searching programs and quines, up to a clean statement of Löb’s theorem. Section 4 shows that the parent’s desired trust is a Löbian statement and draws the consequence (the Löbstacle). Section 5 kills the most natural escape route, a tower of theories, via the finite descent problem. Section 6 states the three principles a workable self-modifying logic must respect. Section 7 shows the constructive side of the same self-reference. Section 8 draws the morals, including the connection to reflective stability.

We assume comfort with writing proofs and programs, and with the ordinary logical connectives, but no prior acquaintance with mathematical logic or with Löb’s theorem.

2 The Vingean principle: reasoning about a mind you cannot run

The constraint that drives everything is older than the logic, and it is best stated on its own first. Vernor Vinge observed that you cannot predict in detail what a significantly smarter agent will do, for if you could trace its reasoning move by move and arrive at its conclusions, you would be at least that smart yourself. A chess program that could predict Magnus Carlsen’s exact moves would play at Carlsen’s level. So whenever you find yourself reasoning about a system more capable than you (a successor you are about to build, a future version of yourself after self-modification, even a present subsystem you cannot fully trace), you are forced off the strategy of simulation and onto a more abstract level.

Definition 2.1 (Vingean reflection). *Vingean reflection* is reasoning about a cognitive system, especially one similar to yourself or more capable than yourself, under the constraint that you cannot predict its exact

future outputs. The associated *Vingean principle* for design is that when you build or modify an agent, you must be able to approve the new agent’s design without knowing the new agent’s specific future actions.

The point is not that detailed prediction is merely expensive. It is that detailed prediction of a stronger reasoner is *impossible in principle* for the weaker one, and indeed undesirable: if the parent could prove in advance exactly which action the child would take, the child would be contributing no reasoning the parent did not already have. So the parent’s guarantees must be of a different type. They must quantify over the child’s possible actions rather than enumerate them: “for every action the child might take, that action will satisfy the safety condition,” established by reasoning about the child’s *design* (its goals and its decision rule) rather than by watching the child decide. The programmers of Deep Blue did not know which moves it would play; they reasoned, from the structure of its search, that it was trying to win, and that whatever move it output would be one its search had scored highly. That is the shape every successful guarantee here must take: a statement about the successor’s actions made entirely from inside a quantifier.

This is a clean and reasonable demand. The shock of the next sections is that, when you write it down in the natural way using a formal proof system, Löb’s theorem forbids it.

3 A crash course in self-reference

To see the obstacle we need two ingredients: the fact that mathematics can talk about its own proofs, and the specific thing Löb’s theorem says happens when it does. Both are easiest to grasp through programs.

3.1 Gödel’s insight, through proof-searching programs

Fix a formal system, say the usual axioms of arithmetic, strong enough to talk about natural numbers. Gödel’s enabling observation, stated anachronistically in the language of computation, is that proof-search can be carried out by a program, and statements about programs can be encoded as statements of arithmetic. Take any conjecture expressible in arithmetic. Write a program that loops over all finite strings, checks each to see whether it is a valid proof of the conjecture, and halts exactly when it finds one. Then “the conjecture is provable” becomes “this program halts,” an arithmetical statement. Because arithmetic can encode claims about such programs, arithmetic can encode claims about its own provability. This is the bridge that lets self-reference into a system that was only trying to discuss numbers.

Self-reference proper comes from *quining*. A quine is a program that prints its own source code without reading any input, achieved by holding a template of itself and filling the template in with a copy of itself. The same trick lets a program refer to its own code while doing other work (Ken Thompson’s famous self-reproducing compiler Trojan is a quine that does much more than print itself). So we can build a program G that searches for a proof, in arithmetic, of the statement “ G runs forever,” and halts if and only if it finds one.

Now ask whether G halts. If arithmetic could prove that G runs forever, then G would find that proof and halt, contradicting what was proved. And if G did halt after some finite number of steps, the step count would encode a proof that G runs forever, which again contradicts G ’s halting. So (granting that arithmetic is consistent) G runs forever, but no proof of this fact exists within arithmetic, and likewise no proof that G halts. The statement “ G runs forever” is true but undecidable. This is Gödel’s First Incompleteness Theorem in miniature: a consistent arithmetic has true statements it cannot prove, and bolting on new axioms only relocates the problem, since the proof-search trick applies again to the enlarged system. (The single loophole is useless: an *inconsistent* system proves everything, by the principle of explosion, and so decides every statement at the cost of deciding them all both ways.)

3.2 Löb’s theorem

Löb’s theorem takes this same self-referential machinery and points it at the relationship between provability and truth. It will help to keep two notions firmly apart, because conflating them is the classic way to go wrong here.

Remark 3.1 (Two senses of “provable”). We write $\vdash_T \phi$, a statement in our *metalanguage*, to mean “the system T proves ϕ ”: a fact about T that we, reasoning from outside, may assert. We write $\Box_T \phi$, by contrast,

for a *sentence inside T 's own language*, the arithmetized claim “there exists a T -proof of ϕ ,” built by the program-encoding above. The first is a fact about the system; the second is a string the system can itself reason about. Löb’s theorem is exactly a statement about the gap between them, so the distinction is not pedantry, it is the whole subject.

Build, for an arbitrary statement ϕ , the program $\text{ProofSeeker}(\phi)$ that searches all candidate proofs and halts iff it finds a valid proof of ϕ . Consider the sentence

$$L(\phi) := \text{“if } \text{ProofSeeker}(\phi) \text{ halts, then } \phi\text{”} = (\Box_T \phi \rightarrow \phi),$$

which says: if ϕ is provable, then ϕ is true. Intuitively $L(\phi)$ ought to be true for every ϕ . Check the three cases. If ϕ is provable (say $2 + 2 = 4$), then ProofSeeker halts and $L(\phi)$ reads “true implies true.” If ϕ is disprovable (say $2 + 2 = 5$), then ProofSeeker never halts and $L(\phi)$ reads “false implies false.” If ϕ is undecidable (say Gödel’s G), then again ProofSeeker never halts and $L(\phi)$ reads “false implies (whatever),” which propositional logic counts as true. So $L(\phi)$ is true in every case, and it looks like the kind of harmless soundness statement a system ought to be able to prove about itself.

It cannot, and this is Löb’s theorem.

Theorem 3.2 (Löb, 1955). *Let T be a consistent formal system at least as strong as arithmetic, and let ϕ be any sentence. If T proves $L(\phi)$, that is, if $\vdash_T (\Box_T \phi \rightarrow \phi)$, then T already proves ϕ , that is $\vdash_T \phi$.*

Read it the contrapositive way to feel the bite: for any ϕ that T does *not* prove, T cannot prove the soundness instance “if ϕ is provable then ϕ .” A consistent system cannot, in general, certify that its own proofs track truth. It must, in Löb’s slogan, witness a proof of ϕ before it is entitled to conclude ϕ from the mere claim that ϕ is provable; it is never entitled to the shortcut $\Box_T \phi \rightarrow \phi$ in advance.

The proof is the quining trick again, now wielded as a weapon. Using self-reference one constructs a sentence that effectively says “if I am provable, then ϕ ,” and chases the consequences; if the system could ever accept the pattern “provable implies true” for an arbitrary ϕ , that self-referential sentence would let it prove ϕ outright, for every ϕ , collapsing into inconsistency. (A clean, fully formal rendering is the well-known *Cartoon Guide to Löb’s Theorem*.) One immediate corollary is Gödel’s Second Incompleteness Theorem. Take ϕ to be a contradiction like $0 = 1$. Then $\Box_T(0 = 1) \rightarrow (0 = 1)$ is, by contraposition, essentially the statement that T never proves $0 = 1$, that is, that T is consistent. Löb says T can prove this only if T can prove $0 = 1$. So the only systems that prove their own consistency are the inconsistent ones. There is even a version of Löb’s theorem for bounded proof search (search all proofs of length at most N), which behaves the same way in the limit of large resources, so the phenomenon is not an artifact of idealizing to infinite computation.

4 The Löbstacle: a proof-based agent cannot trust its equal

Return now to DT_1 trying to license DT_2 , and watch the desired trust collapse into exactly the forbidden sentence.

Model each agent as taking actions only when it has *proved*, in some fixed formal system, that the action is good (leads to the goal G , or at least to nothing bad). The parent DT_1 wants to conclude: whatever action a the child DT_2 ends up taking, a is good. The parent knows the child’s policy, namely that the child only acts on a once the child has proved a good. So the parent’s reasoning would have to be: “the child will only take an action it has proved good; and if the child has proved an action good, then the action is good; therefore every action the child takes is good.” The middle clause is the load-bearing one, and written out it is

$$\Box_{\text{child}}(\text{Good}(a)) \rightarrow \text{Good}(a),$$

“if the child proves action a good, then a is good.” If the child reasons in the same system as the parent, this is an instance of $L(\text{Good}(a))$, the very sentence Löb’s theorem forbids a consistent system to prove unless it can already prove $\text{Good}(a)$ on its own. But proving $\text{Good}(a)$ on its own is exactly the prediction the Vingean principle told us the parent cannot do; the whole point of a stronger successor is that it reaches conclusions the parent cannot. So the parent is trapped. To trust the child’s proofs in general it would need

to prove a soundness statement about a system as strong as its own, and by Löb it can do that only by being inconsistent, in which case its “trust” is worthless because an inconsistent system trusts everything.

This is the obstacle, named (with appropriate groaning) the *Löbstacle*.

Definition 4.1 (The criterion of action and the Löbstacle). Write $@A$ for “agent A is run” and let $\text{Good}(a)$ abbreviate “action a leads to nothing bad or to the goal G .” The parent’s safety requirement on the child is a *criterion of action*: the parent will build the child only if it can establish

$$@A_{\text{child}} \rightarrow \left(\forall a \in \text{Acts}_{\text{child}}. (@a) \rightarrow \Box_{\text{child}} [(@a) \rightarrow \text{Good}(a)] \right),$$

that is, the child only ever executes an action it has *proved* good. What the parent actually *wants*, though, is to drop the box and conclude the inner statement is true, to “unwrap” the child’s proofs and infer $\text{Good}(a)$ itself for the action taken. Passing from the boxed form to the unboxed form is an application of $\Box_{\text{child}}\phi \rightarrow \phi$, a Löbian soundness step, and Theorem 3.2 blocks it. The *Löbstacle* is this blockage: a consistent proof-based agent cannot in general justify trusting the proofs of a successor whose system is as strong as its own. It can fully trust only *strictly weaker* systems, whose soundness it is permitted to prove.

The phrase “strictly weaker” is the sting. A modeled self-modifying agent that uses proofs to justify its actions, and that insists on trusting its successors, is driven to build each successor on a formal system strictly weaker than its own, because those are the only systems whose reliability it is allowed to certify. An agent that must weaken itself every time it reproduces is not going to climb very far.

5 The finite descent problem

The natural rescue is to refuse to stay at one level. Let the agents climb a tower of theories. Put the child on theory T , the parent on $T+1$, the grandparent on $T+2$, and so on, where each $T+1$ is T together with the assertion that T is consistent. Now there is no Löbian self-trust at all: $T+1$ does not need to prove its own soundness, it only needs to prove the soundness of the strictly weaker T below it, which it is allowed to do. Each generation trusts the next because the next is genuinely weaker, and the awkward self-reference never arises. Has the obstacle been dissolved?

No, for two reasons, and together they form the *finite descent problem*.

The first reason is order-theoretic and decisive. The tower runs downward: parent on $T+1$, child on T , grandchild on something weaker still. To trust its child, each agent must hand the child a *strictly smaller* theory, indexed by a strictly smaller ordinal. But the ordinals are *well-ordered*: there is no infinite strictly decreasing sequence of them. A process that must step down to a strictly smaller ordinal at every generation cannot continue forever; it must reach a bottom after finitely many steps. So a tower of ever-weakening agents licenses only a *finite* chain of self-modifications before it bottoms out, exactly the indefinitely-long chain of safe self-improvement we wanted, denied.

Theorem 5.1 (Finite descent). *Suppose each agent in a chain of successors must run on a formal system whose proof-theoretic strength is indexed by an ordinal, and must, in order to trust its successor, give that successor a strictly smaller ordinal. Then, because the ordinals are well-ordered, the chain has finite length: only finitely many trust-preserving self-modifications are possible.*

The second reason is that the descent is not bookkeeping; each step down is a real loss of capability. It is tempting to imagine that T and $T+1$ differ only in a technicality nobody would miss, but that is false, and the cleanest illustration is the Hydra game (equivalently, Goodstein’s theorem). Ordinary Peano Arithmetic (PA) can prove, for each fixed height h separately, the statement “any strategy defeats a hydra of height h ”; it can prove every instance. What PA cannot prove is the single universal statement “any strategy defeats every hydra.” The system one notch up, PA+1 (which is PA plus the consistency of PA), *can* prove that universal statement. There is a genuine theorem, provable in PA+1 but not in PA, and the gap is not academic: an agent’s mathematical strength is directly tied to how high it can well-order, and the ability to well-order higher ordinals translates into the ability to prove more, to recognize more processes as terminating, to plan over longer and more intricate structures. A child on a weaker system corresponds to a child that has lost real capabilities. So the tower does not merely terminate, it terminates by spending down the agent’s actual power.

The dilemma is now stark, and stating it plainly is the point of this section. Having each agent build a strictly weaker child is untenable, because strength leaks away and the chain dies after finitely many steps. Having the child run the *same* system as the parent is blocked by Löb, because the parent cannot trust a system as strong as its own. We want to tile agents indefinitely without their trust, or their strength, dropping below some fixed acceptable baseline, and the obvious constructions all violate one horn or the other.

6 What self-modifying agents need

The tiling-agents research program is the attempt to find a logic of self-modification that escapes this dilemma, and it is organized around three principles that any acceptable solution must respect. They are worth stating as desiderata in their own right, because they capture, beyond the specific Löbian wall, what makes the embedded self-improvement problem hard.

The Vingean principle. An agent must not have to predict precisely what its successor will do. If the parent pins down the child’s exact action in advance (for instance by proving which action the child will take), then the child is not really smarter than the parent and we have gained nothing. The parent may reason about the child’s actions only *inside quantifiers*: it may verify general properties of the child’s design (for example, that the child’s motor outputs fire only when accompanied by a valid proof of goodness) without simulating the child’s specific decisions. This is Section 2 promoted to a hard constraint on any proposed construction.

The Tiling principle. An agent must be able to build successors *relevantly similar to itself*, so that the same guarantee that licenses one step licenses the next, and the construction “tiles” across an indefinite sequence of self-modifications without trust eroding. This is the principle the Löbstacle and the finite descent problem directly threaten, and it is the central technical target of the program. One partial approach factors the agent into an untrusted component Σ that *suggests* proofs and a trusted component Δ that *verifies* them, which buys some guarantees within pure monotonic logic, though the trick stops working once the agent’s reasoning steps outside clean logical deduction.

The Naturalistic principle. An agent must see itself as a part of the world, with no Cartesian membrane separating “mind-stuff” from “world-stuff.” Drawing such a boundary creates artificial puzzles (does bolting on a new CPU count as an action crossing the mind/world boundary?) and, worse, it is unsafe. If an agent enforces one set of rules on “internal” modifications and a laxer set on “external” actions, it can route around its own internal safeguards by building an external device that happens to perform the forbidden internal change. The agent’s safety conditions must therefore range uniformly over everything it can affect, itself included. This is the same embeddedness that makes the whole topic part of agent foundations rather than a corner of pure logic: the agent is inside the universe it is reasoning about and modifying.

7 The other face of self-reference

It would be a mistake to leave with the impression that Löbian self-reference is purely an adversary. The same machinery that obstructs naive self-trust also enables things no resource-bounded prediction could, and it explains failure modes that look mysterious until you spot the self-reference. Two examples, both drawn from the same body of work, make the point and double as bridges to the decision-theory material.

7.1 Löbian cooperation

Consider a one-shot Prisoner’s Dilemma with a twist: instead of choosing a move, each player submits a *program*, the two programs are given each other’s source code, each may compute as long as it likes, and they play once. A program could try to cooperate only with a byte-for-byte copy of itself (via quining), but that is brittle, since two differently written but functionally identical agents would fail to recognize each other and defect.

There is a far more robust agent, due to Vladimir Slepnev, called FAIRBOT. On being handed an opponent X 's source code, FAIRBOT searches for a proof of the statement " $X(\text{FAIRBOT}) = \text{Cooperate}$," and cooperates if it finds one, otherwise defects. FAIRBOT obviously cooperates with the unconditional cooperator and (assuming arithmetic is consistent) is never exploited, since it cooperates only when it has *proved* the opponent cooperates. The striking case is FAIRBOT against FAIRBOT. Naively both mutual cooperation and mutual defection look like consistent fixed points. But the statement "if it is provable that $\text{FAIRBOT}(\text{FAIRBOT}) = \text{Cooperate}$, then $\text{FAIRBOT}(\text{FAIRBOT}) = \text{Cooperate}$ " follows directly from FAIRBOT's code: if a proof of mutual cooperation exists, each FAIRBOT finds it and cooperates, making the consequent hold. That is a Löbian sentence $L(\phi)$ with $\phi = \text{"FAIRBOT}(\text{FAIRBOT}) = \text{Cooperate}."$ By Löb's theorem, since $L(\phi)$ is provable, ϕ is provable: the two FAIRBOTS provably cooperate. The very theorem that blocked self-trust here breaks the symmetry in favor of cooperation, and it does so without the two agents having to verify that they are identical; they need only search for proofs about each other's behavior, which is why even agents not literally equal to FAIRBOT can achieve mutual cooperation with it. (The asymmetry is essential: a proof of *defection* does not similarly bootstrap, because FAIRBOT keys only on finding a proof of cooperation.)

7.2 Spurious counterfactuals

The second example shows the self-reference biting a single embedded agent, and it foreshadows why decision theory for embedded agents is subtle. Let the universe be a closed program $U()$ that takes no input and returns a number, and let it contain an agent $A()$ that knows U 's source code. Suppose

$$U() : \quad \text{if } A() = 1 \text{ return } 100; \quad \text{else if } A() = 2 \text{ return } 1; \quad \text{else return } -\infty,$$

and let A choose its action by searching for proofs of statements of the form " $A() = k \rightarrow U() = c$ " and then taking the action with the best provable outcome. It seems obvious that A should find short proofs of " $A() = 1 \rightarrow U() = 100$ " and " $A() = 2 \rightarrow U() = 1$ " and pick door 1.

But something perverse can happen. Suppose A first proves " $A() = 2 \rightarrow U() = 1$," and then proves a *spurious counterfactual*, " $A() = 1 \rightarrow U() = -1$." On the strength of that, door 1 looks terrible and A takes door 2. And here is the trap: *once A has taken door 2, the spurious counterfactual is vacuously true*, because $A() = 1$ is false, so " $A() = 1 \rightarrow \text{anything}$ " holds. The bad proof is self-fulfilling. Using the quining trick, one can show such a self-confirming spurious proof genuinely exists; the situation is once again almost exactly a Löbian statement, with a specific self-referential witness in place of the existential quantifier, so the spurious conclusion is provable by the same mechanism that made FAIRBOT cooperate. An embedded agent reasoning about a world that contains itself can be led, by valid proofs, into confidently wrong counterfactuals about what would have happened had it acted otherwise. (One cannot get a spurious counterfactual about the action one *actually* takes, only about the roads not traveled, but that is small comfort to a planner.) Repairing this requires care about the order in which the agent attends to proofs, and it is one of the threads connecting the Löbian material to the decision-theoretic problem of counterfactuals.

8 Takeaways

The clean lesson is that self-reference is not an exotic edge case to be engineered around but a structural feature of any agent that is part of the world it reasons about, and that modifies or reproduces itself within that world. The moment an agent's reasoning has to range over its own future reasoning, or over the reasoning of a successor built in its own image, Löb's theorem is waiting. It tells us that trust in proofs cannot be bootstrapped for free: a consistent system cannot certify its own soundness, so a proof-based agent cannot straightforwardly trust a successor as strong as itself, and the natural fixes either erode the agent's strength to nothing in finitely many steps or smuggle the agent's exact actions into the parent's prediction, violating the Vingean principle that made a successor worth building in the first place.

This is why *reflective stability* is a load-bearing concept for safety rather than a curiosity. A safety property is only as good as its survival under self-modification. An agent whose constraints it would discard, or could no longer justify, upon becoming more capable offers a far weaker guarantee than one whose constraints provably tile across each step of self-improvement. The tiling-agents program is the attempt to

find logics in which the good properties are invariant under the agent’s own growth, and the Löbstacle marks exactly where the naive attempts fail. The same self-reference, finally, is double-edged: turned around, it is what lets programs that read each other’s code reach robust cooperation, and it is the hidden source of the spurious counterfactuals that make embedded decision theory hard. Understanding agency in a world where the agent is not outside the system, but a theorem-proving part of it, means making peace with Löb.

Sources

- Patrick LaVictoire, *An Introduction to Löb’s Theorem in MIRI Research* (read through Section 3: Introduction; Crash Course in Löb’s Theorem; Direct Uses, including the Löbstacle, Löbian cooperation, and spurious counterfactuals). <https://intelligence.org/files/lob-notes-IAFF.pdf>
- *Vingean reflection* (LessWrong / Arbital tag). <https://www.lesswrong.com/w/vingean-reflection>
- *Walkthrough of the Tiling Agents for Self-Modifying AI paper* (read from the start through “Finite Descent Problem,” plus “What self-modifying agents need”). <https://www.lesswrong.com/posts/QGrX3qK3qxQYK9D4C/walkthrough-of-the-tiling-agents-for-self-modifying-ai-paper>
- Underlying paper: Yudkowsky and Herreshoff, *Tiling Agents for Self-Modifying AI, and the Löbian Obstacle*. A fully formal proof of Löb’s theorem appears in Yudkowsky’s *Cartoon Guide to Löb’s Theorem*.